Practice Set: 09

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define an **Array** in C++ and explain why it is historically called a "subscripted variable". Contrast the scalar variable method against the array method when handling data for 100 students.

Question 2: Explain the concept of **Pointer Decay** and contiguous memory layout in C++ arrays. State the pointer arithmetic formula used by the compiler to locate element a[i] given a base address in RAM.

Question 3: What are the critical rules for **1-D Array Declarations** regarding standalone omitted sizes, uninitialized local garbage values, array overflow, and partial initialization?

Question 4: Explain what happens in modern C++ (C++11) when an array is declared using uniform list initialization syntax like int a[5]{};.

Question 5: What is the **Out-of-Bounds Indexing Bug** in C++ raw arrays? Explain the runtime consequences of accessing an index beyond the declared array boundary.

Question 6: Explain the structure of a **Two-Dimensional (2-D) Array** in C++. State the mandatory rule regarding dimension specification when declaring and initializing a 2-D array.

Question 7: Describe **Row-Major Order** memory layout for 2-D arrays in physical RAM. Draw or map how a 2-row by 3-column matrix is placed into linear memory addresses.

Question 8: Explain the concept of a **Three-Dimensional (3-D) Array** in C++ using the "pages/layers" analogy. State the declaration syntax and explain the role of each nested loop required to traverse 3D space.

Question 9: Compare **C-Style Raw Arrays**, **std::array<T, N>**, and **std::vector<T>** across the following features: Memory Allocation Location, Sizing Mutability, Self-Size Tracking (.size()), and Safe Bounds-Checked Access (.at()).

Question 10: Write a complete C++ program that initializes a 2-D matrix (int a[2][3]), prints its elements using nested loops, demonstrates safe access with std::array, and dynamically appends elements to a std::vector using push_back().

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Does the array identifier name in C++ allocate separate pointer storage in RAM or act as a constant pointer to the base address?

Question 12 (Viva): If an array int a[5] = {10, 50}; is initialized partially, what values are assigned to slots a[2], a[3], and a[4]?

Question 13 (Viva): Why is int arr[2][] = {{1, 2}, {3, 4}}; an invalid array declaration in C++?

Question 14 (Viva): Does C++ perform automatic runtime bounds-checking when accessing elements of raw C-style arrays using the subscript operator []?

Question 15 (Viva): Where in physical memory are elements of std::array allocated compared to std::vector?

Question 16 (Viva): What is the result of using .at(index) on a std::array or std::vector if the index provided is out of bounds?

Question 17 (Viva): In what order are elements of multi-dimensional arrays laid out in physical RAM in C++?

Question 18 (Viva): What is the exact element capacity of a 3-D array declared as int arr[2][3][2]?

Question 19 (Viva): Why does declaring a standalone raw array like int a[]; inside a function body cause a compilation error?

Question 20 (Viva): Why is std::array preferred over raw C-style arrays in modern C++ software design?

Answers & Explanations

Answer: 01

Definition: An Array is a linear collection of homogeneous data elements stored in strictly contiguous memory locations. It is referred to as a subscripted variable because individual elements are referenced by an integer subscript or index.

Scalar vs. Array Method: Managing 100 scalar variables (int n1, n2... n100;) creates unmaintainable, error-prone code that cannot be cleanly iterated over. The array method (int numbers[100];) allocates one continuous block where all elements share a single base name and are indexed with simple offsets.

Answer: 02

Pointer Decay & Contiguity: Array elements reside side-by-side in physical RAM. The array name decays into a constant pointer holding the base address of element 0.

Pointer Arithmetic Formula:

Address of Element a[i] = Base Address + i *sizeof(data_type)

For example, if int a[5] starts at 0x1000, a[1] is located at 0x1000 + (1 * 4) = 0x1004.

Answer: 03

Standalone Size Omission: Omitting size brackets without immediate initializer values (e.g., int a[];) triggers a compilation error.

Garbage Values: Uninitialized local arrays declared inside function scopes contain random bit patterns ("garbage values").

Array Overflow: Providing more initializer elements than the stated size (e.g., int a[5] = {10, 20, 30, 40, 50, 60};) causes a compile error.

Partial Initialization: Supplying fewer initializers than the declared capacity causes the compiler to automatically zero-fill the remaining uninitialized slots.

Answer: 04

In C++11, direct brace initialization with empty braces (int a[5]{};) performs uniform zero-initialization, ensuring every slot in the array is explicitly set to 0 without garbage bits.

Answer: 05

Definition: C++ does not perform automatic runtime bounds-checking on raw C-style arrays.

Consequences: Accessing out-of-bounds indices (e.g., index 10 on a 5-element array) attempts to read or overwrite adjacent stack/heap memory, corrupting data or triggering segmentation faults.

Answer: 06

2-D Array Structure: A 2-D array is structurally an "array of arrays" that maps data logically into horizontal rows and vertical columns (data_type name[rows][columns]).

Dimension Specification Rule: When declaring and initializing a 2-D array, specifying the column dimension is strictly mandatory so the compiler knows

the row length; the row dimension is optional (e.g., int arr[][3] = {...} is valid, but int arr[2][] = {...} is illegal).

Answer: 07

Row-Major Order: RAM is a 1-D linear memory space. In Row-Major order, Row 0 is stored sequentially, followed immediately by Row 1, Row 2, etc.

Memory Map for int a[2][3]:

Logical Matrix: [[5, 8, 9], [10, 12, 11]]

Linear Memory Layout: a[0][0] (5) -> a[0][1] (8) -> a[0][2] (9) -> a[1][0] (10) -> a[1][1] (12) -> a[1][2] (11).

Answer: 08

3-D Array Concept: A 3-D array is an "Array of 2-D Arrays," conceptualized as a multi-page book of spreadsheet grids (data_type name[pages][rows][columns]).

Nested Loop Traversal:

Loop 1 (Outer): Iterates through page/depth layers.

Loop 2 (Middle): Iterates through rows on the active page.

Loop 3 (Inner): Iterates through individual column elements within the active row.

Answer: 09

Feature	**C-Style Raw Array**	**std::array<T, N>**	**std::vector<T>**
Memory Location	Stack	Stack	Dynamic Heap
Sizing Mutability	Fixed at compile time	Fixed at compile time	Dynamic runtime resizing
Knows Its Size	No (Decays to pointer)	Yes (via .size())	Yes (via .size())
Safe Access	No bounds checking	Yes (via .at())	Yes (via .at())

Answer: 10

```
#include <iostream>
#include <array>
#include <vector>
int main()
{
   // 1. 2-D Array Matrix Navigation
   int a[2][3] = {
      {5, 8, 9},
      {10, 12, 11}
   };

   std::cout << "2-D Array Matrix:" << std::endl;
   for (int i = 0; i < 2; i++)
   {
      for (int j = 0; j < 3; j++)
       {

         std::cout << a[i][j] << " ";
      }
      std::cout << std::endl;
   }

   // 2. Fixed Stack Array with std::array

   std::array<int, 3> modernArr = {10, 20, 30};
   std::cout << "\nstd::array size: " << modernArr.size() << std::endl;
   std::cout << "Safe element at index 1: " << modernArr.at(1) << std::endl;

   // 3. Dynamic Heap Array with std::vector
   std::vector<int> dynamicVec = {100, 200};
   dynamicVec.push_back(300);

   std::cout << "Dynamic Vector items: ";
   for (int val : dynamicVec)
   {
      std::cout << val << " ";
   }
   std::cout << std::endl;

   return 0;
}
```

Answer: 11

The array name decays to a pointer value representing the base address; it does not allocate separate pointer variable storage in RAM.

Answer: 12
They are automatically zero-filled by the compiler, setting slots a[2], a[3], and a[4] to 0.

Answer: 13
Because specifying the second dimension (columns) is mandatory in C++ so the compiler can compute row offsets in contiguous memory.

Answer: 14
No, raw C-style arrays do not perform bounds checking with [].

Answer: 15
std::array allocates its fixed buffer directly on the **Stack**, whereas std::vector stores elements on the **Dynamic Heap**.

Answer: 16
It performs runtime bounds verification and throws an out-of-range exception if the requested index exceeds container limits.

Answer: 17
In **Row-Major Order** (row elements stored completely one after another).

Answer: 18
Total capacity is $2 \times 3 \times 2 = 12$ integer elements.

Answer: 19
The compiler cannot determine how much RAM space to allocate without an explicit size or an initializer list.

Answer: 20
It provides pointer safety, prevents unintentional pointer decay, tracks its own size (.size()), and provides safe bounds checking (.at()) with zero runtime overhead over raw arrays.